

## Пузырьковая сортировка (улучшенная)

Представь ряд карточек с числами. Идея — как пузырьки воздуха в воде: тяжёлые «всплывают» в конец.

Ты идёшь по ряду **слева направо и сравниваешь соседей парами**. Если левый больше правого — меняешь их местами. Дошёл до конца — самое большое число «утонуло» (всплыло) в самый конец, оно теперь на своём месте. Повторяешь проход заново — теперь второе по величине встанет на предпоследнее место. И так, пока не отсортируешь весь ряд.

Один проход по [5, 1, 4, 2]:

[5, 1, 4, 2] сравни 5 и 1 →  $5 > 1$ , меняем → [1, 5, 4, 2]

[1, 5, 4, 2] сравни 5 и 4 →  $5 > 4$ , меняем → [1, 4, 5, 2]

[1, 4, 5, 2] сравни 5 и 2 →  $5 > 2$ , меняем → [1, 4, 2, 5]

конец прохода: 5 встало на место 

**В чём «улучшение».** Наивная версия всегда делает все проходы, даже если массив уже отсортирован. Умная — заводит флажок: «а был ли хоть один обмен за проход?». Если за целый проход **ни одного обмена не случилось** — значит всё уже упорядочено, и можно сразу выходить, не тратя оставшиеся проходы.

```
java
static void bubbleSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        boolean swapped = false;           // флажок: были ли обмены
        for (int j = 0; j < n - 1 - i; j++) { // -i: хвост уже отсортирован
            if (a[j] > a[j + 1]) {
                int tmp = a[j];             // меняем соседей местами
                a[j] = a[j + 1];
                a[j + 1] = tmp;
                swapped = true;
            }
        }
        if (!swapped) break;               // проход без обменов → всё готово
    }
}
```

Два трюка тут: **swapped** (ранний выход на почти отсортированных данных) и  $n - 1 - i$  (не трогаем уже всплывший хвост — он на местах). Из-за флажка на **уже отсортированном** массиве работает за  $O(n)$  — один проход и выход. В худшем случае (массив задом наперёд) всё равно  $O(n^2)$ .

## Сортировка выбором

Другая идея, попроще для понимания. Ты **выбираешь минимум** из всего, что осталось, и ставишь его в начало.

Разделяешь ряд мысленно на две части: слева «уже отсортировано», справа «ещё бардак». На каждом шаге проходишь весь бардак, находишь там **самое маленькое число** и меняешь его с первым элементом бардака. Граница отсортированной части сдвигается на один вправо. Повторяешь.

[5, 1, 4, 2]:

весь массив бардак → минимум = 1 → меняем с позицией 0

[1 | 5, 4, 2] отсортировано: [1]

бардак [5,4,2] → минимум = 2 → меняем с позицией 1

[1, 2 | 4, 5] отсортировано: [1, 2]

бардак [4,5] → минимум = 4 → уже на месте

[1, 2, 4 | 5] готово ✓

java

```
static void selectionSort(int[] a) {
    int n = a.length;
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;           // считаем, что минимум — на позиции i
        for (int j = i + 1; j < n; j++) { // ищем настоящий минимум справа
            if (a[j] < a[minIndex]) {
                minIndex = j;       // нашли меньше — запоминаем где
            }
        }
        int tmp = a[i];             // ставим минимум в начало бардака
        a[i] = a[minIndex];
        a[minIndex] = tmp;
    }
}
```

**Разница между ними, чтобы не путать.** Пузырёк меняет соседей много раз за проход и умеет рано выходить. Выбор находит минимум и делает **всего один обмен за проход**, но выхода пораньше у него нет — он всегда честно проходит  $O(n^2)$ , даже на готовом массиве. Пузырёк «переставляет по чуть-чуть», выбор «нашёл и переставил разом».

8. Consider the following class.

```
/** A class that sorts an array of Integer objects from
 * largest to smallest using a selection sort.
 */
public class Sorter
{
    private Integer[] a;

    public Sorter(Integer[] arr)
    { a = arr; }

    /** Swap a[i] and a[j] in array a. */
    private void swap(int i, int j)
    { /* implementation not shown */ }

    /** Sort array a from largest to smallest using selection sort.
     * Precondition: a is an array of Integer objects.
     */
    public void selectionSort()
    {
        for (int i = 0; i < a.length - 1; i++)
        {
            //find max element in a[i+1] to a[n-1]
            Integer max = a[i];
            int maxPos = i;
            for (int j = i + 1; j < a.length; j++)
                if (max.compareTo(a[j]) < 0) //max less than a[j]
                {
                    max = a[j];
                    maxPos = j;
                }
            swap(i, maxPos);    //swap a[i] and a[maxPos]
        }
    }
}
```

If an array of Integer contains the following elements, what would the array look like after the third pass of selectionSort, sorting from high to low?

89 42 -3 13 109 70 2

- (A) 109 89 70 13 42 -3 2
- (B) 109 89 70 42 13 2 -3
- (C) 109 89 70 -3 2 13 42
- (D) 89 42 13 -3 109 70 2
- (E) 109 89 42 -3 13 70 2